

TALLER BANCO ANDINO

1- Ingeniería del dominio con ia

Solución de la IA:

Fase 5:

5.1:

● Nivel Básico (Problemas 1 a 10)

Problema 1: Listado de Clientes Activos (Persona Natural)

- **Objetivo de negocio:** Obtener el directorio comercial de personas naturales activas para una campaña de fidelización digital.
- **Tablas necesarias:** `banco_core.cliente`, `banco_core.persona_natural`, `banco_core.estado_cliente`.
- **Conceptos SQL evaluados:** SELECT, WHERE, JOIN básico, alias de tablas y proyección de columnas.
- **Criterio de validación:** Debe retornar únicamente los clientes cuyo estado sea 'ACTIVO' y su `tipo_cliente` sea 'NATURAL', mostrando su código de cliente, número de documento y nombre completo.

Problema 2: Conteo de Cuentas por Estado

- **Objetivo de negocio:** Monitorear la distribución operativa de las cuentas del banco (activas, bloqueadas, cerradas, pendientes).
- **Tablas necesarias:** `banco_core.cuenta`, `banco_core.estado_cuenta`.
- **Conceptos SQL evaluados:** GROUP BY, agregación COUNT(), JOIN.
- **Criterio de validación:** Debe presentar cada código y descripción del estado de la cuenta junto con la cantidad total de cuentas registradas en dicho estado.

Problema 3: Cuentas Corrientes con Límite de Sobregiro Autorizado

- **Objetivo de negocio:** Identificar a los clientes preferenciales o empresariales a los que se les ha otorgado un cupo de sobregiro activo.
- **Tablas necesarias:** `banco_core.cuenta`, `banco_core.producto`.
- **Conceptos SQL evaluados:** Filtros numéricos (WHERE >), JOIN, ordenamiento (ORDER BY DESC).

- **Criterio de validación:** Filtrar únicamente productos de tipo 'CORRIENTE' donde el limite_sobregiro sea mayor a 0.00, ordenando de mayor a menor cupo.

Problema 4: Transacciones Financieras de un Período Específico

- **Objetivo de negocio:** Auditar el flujo de operaciones financieras en estado contabilizado (POSTED) procesadas en un rango de fechas.
- **Tablas necesarias:** banco_core.transaccion, banco_core.tipo_transaccion, banco_core.estado_transaccion.
- **Conceptos SQL evaluados:** Manejo de tipos de datos temporales (TIMESTAMPZ), BETWEEN / operadores de comparación, filtros con múltiples condiciones.
- **Criterio de validación:** Mostrar el ID de transacción, tipo de transacción, monto y la fecha exacta para todas las operaciones en estado 'POSTED' ejecutadas entre dos fechas dadas.

Problema 5: Red de Oficinas Bancarias por Municipio

- **Objetivo de negocio:** Generar el inventario de la infraestructura física del banco ordenado por la división político-administrativa DANE.
- **Tablas necesarias:** banco_core.oficina, banco_core.municipio, banco_core.departamento.
- **Conceptos SQL evaluados:** INNER JOIN encadenado de tres tablas, selección de campos explícitos.
- **Criterio de validación:** Listar el código de oficina, nombre de la oficina, dirección, municipio y departamento correspondiente.

Problema 6: Promedio y Monto Máximo de Consignaciones

- **Objetivo de negocio:** Establecer el perfil general del volumen monetario aportado mediante depósitos y consignaciones.
- **Tablas necesarias:** banco_core.transaccion, banco_core.tipo_transaccion.
- **Conceptos SQL evaluados:** Funciones de agregación (AVG(), MAX()), redondeo (ROUND()).
- **Criterio de validación:** Calcular el valor promedio (redondeado a 2 decimales) y el monto máximo registrado para transacciones de tipo 'CONSIGNACION' que estén en estado POSTED.

Problema 7: Directorio de Contactos Principales de Clientes

- **Objetivo de negocio:** Extraer la base de datos de correos y teléfonos para el envío masivo de extractos bancarios al cierre de mes.

- **Tablas necesarias:** `banco_core.cliente`, `banco_core.cliente_contacto`.
- **Conceptos SQL evaluados:** Filtros sobre valores booleanos (`es_principal IS TRUE`), proyección sin duplicados.
- **Criterio de validación:** Retornar el código del cliente, dirección de residencia, teléfono principal y correo electrónico, garantizando 1 solo registro por cliente.

Problema 8: Cuentas Aperturadas en la Oficina Principal

- **Objetivo de negocio:** Evaluar el nivel de captación de la sucursal matriz respecto a la red nacional.
- **Tablas necesarias:** `banco_core.cuenta`, `banco_core.oficina`.
- **Conceptos SQL evaluados:** Filtro por código o nombre de oficina, `COUNT()`.
- **Criterio de validación:** Retornar la cantidad total de cuentas cuyo `id_oficina_apertura` corresponda a la oficina con código 'OFC-001'.

Problema 9: Operaciones de Alto Valor (Reporte Prevención Lavado de Activos)

- **Objetivo de negocio:** Generar el reporte de operaciones individuales que superen un umbral regulatorio para el oficial de cumplimiento (SARLAFT).
- **Tablas necesarias:** `banco_core.transaccion`.
- **Conceptos SQL evaluados:** Filtro por umbral numérico (`monto > 10000000.00`), selección de campos clave (`idempotency_key`, `monto`).
- **Criterio de validación:** Mostrar todas las transacciones contabilizadas cuyo monto sea estrictamente mayor a \$10.000.000 COP, ordenadas de mayor a menor monto.

Problema 10: Frecuencia de Eventos Administrativos de Cuenta

- **Objetivo de negocio:** Identificar los trámites sobre contratos de cuenta más demandados por los clientes en la red de oficinas.
- **Tablas necesarias:** `banco_core.evento_cuenta`, `banco_core.tipo_evento_cuenta`.
- **Conceptos SQL evaluados:** `GROUP BY`, `ORDER BY DESC`.
- **Criterio de validación:** Listar el código y nombre del tipo de evento administrativo junto con el número total de veces que ha sido registrado en la tabla `evento_cuenta`.

🌀 Nivel Intermedio (Problemas 11 a 20)

Problema 11: Saldo Total Consolidado por Cliente

- **Objetivo de negocio:** Determinar el pasivo financiero total que representa cada cliente para el banco mediante la suma del saldo de sus cuentas activas.
- **Tablas necesarias:** `banco_core.cliente`, `banco_core.cuenta_titular`, `banco_core.cuenta`.
- **Conceptos SQL evaluados:** JOIN en modelos N:M, agregación `SUM()`, `GROUP BY`, control de nulos mediante `COALESCE()`.
- **Criterio de validación:** Devolver el código de cliente y la suma total de `saldo_disponible`. Se debe evitar la duplicación de saldos en cuentas compartidas por múltiples titulares.

Problema 12: Clientes con Más de N Cuentas Registradas

- **Objetivo de negocio:** Identificar clientes multiproducto para programas de fidelización o análisis de concentración de riesgo.
- **Tablas necesarias:** `banco_core.cliente`, `banco_core.cuenta_titular`.
- **Conceptos SQL evaluados:** Agregación con filtro post-agregado (`HAVING COUNT(*) > X`).
- **Criterio de validación:** Listar el código del cliente y la cantidad de cuentas vinculadas activas, únicamente para aquellos clientes que posean más de 3 cuentas.

Problema 13: Cuentas Inactivas Sin Movimientos Financieros

- **Objetivo de negocio:** Detectar cuentas aperturadas que no han registrado ninguna transacción contable en el ledger para aplicar cobros de mantenimiento o marcar inactividad.
- **Tablas necesarias:** `banco_core.cuenta`, `banco_core.transaccion`.
- **Conceptos SQL evaluados:** LEFT JOIN con condición `IS NULL`, o subconsulta con `NOT EXISTS`.
- **Criterio de validación:** Mostrar las cuentas en estado activa que no figuren como `id_cuenta_origen` ni como `id_cuenta_destino` en la tabla `transaccion`.

Problema 14: Flujo de Transferencias Inter-Regionales

- **Objetivo de negocio:** Analizar la movilidad del dinero entre cuentas pertenecientes a oficinas de diferentes departamentos de Colombia.
- **Tablas necesarias:** `banco_core.transaccion`, `banco_core.cuenta` (origen y destino), `banco_core.oficina`, `banco_core.municipio`, `banco_core.departamento`.

- **Conceptos SQL evaluados:** Múltiples instancias de la misma tabla mediante alias, comparación cruzada de llaves geográficas.
- **Criterio de validación:** Mostrar el ID de transacción, monto, departamento de origen y departamento de destino, garantizando que el departamento de origen sea distinto al departamento de destino.

Problema 15: Balanza de Comprobación Contable (Plan Único de Cuentas - PUC)

- **Objetivo de negocio:** Generar el informe de reconciliación agregada de débitos y créditos del libro mayor por cada cuenta PUC.
- **Tablas necesarias:** banco_core.asiento_contable, banco_core.plan_cuentas.
- **Conceptos SQL evaluados:** Agregación condicional (SUM(CASE WHEN ...)), comprobación de partida doble.
- **Criterio de validación:** Presentar el código PUC, nombre de la cuenta contable, total de movimientos DÉBITO, total de movimientos CRÉDITO y la diferencia (Saldo Neto).

Problema 16: Clientes Persona Jurídica con sus Representantes Legales Activos

- **Objetivo de negocio:** Mapear la estructura de representación legal de los clientes corporativos para firma de contratos y autorizaciones.
- **Tablas necesarias:** banco_core.persona_juridica, banco_core.historico_representante_legal, banco_core.persona_natural.
- **Conceptos SQL evaluados:** INNER JOIN de entidades extendidas, filtrado por flags activos (activo IS TRUE) y rangos de fecha vigentes.
- **Criterio de validación:** Listar el NIT, Razón Social, Cédula del Representante Legal, Nombre Completo y Fecha de Inicio de la representación legal.

Problema 17: Top 5 Municipios con Mayor Volumen Dinámico Transaccionado

- **Objetivo de negocio:** Identificar las plazas comerciales con mayor movimiento económico para justificar apertura de nuevas oficinas.
- **Tablas necesarias:** banco_core.transaccion, banco_core.cuenta, banco_core.oficina, banco_core.municipio.
- **Conceptos SQL evaluados:** SUM(monto), GROUP BY, ORDER BY DESC, LIMIT.
- **Criterio de validación:** Devolver los 5 municipios que acumulen la mayor suma en dinero por transacciones donde la cuenta de origen pertenezca a una oficina de dicho municipio.

Problema 18: Cuentas con Alertas de Bloqueos Frecuentes

- **Objetivo de negocio:** Detectar cuentas que hayan sufrido múltiples bloqueos administrativos o judiciales en su ciclo de vida.
- **Tablas necesarias:** `banco_core.cuenta`, `banco_core.evento_cuenta`, `banco_core.tipo_evento_cuenta`.
- **Conceptos SQL evaluados:** Expresiones de Tabla Comunes (CTEs), agregación con filtro de tipos de evento.
- **Criterio de validación:** Listar el número de cuenta y la cantidad de eventos de tipo 'BLOQUEO_PARCIAL' o 'BLOQUEO_TOTAL', mostrando solo aquellas con más de 1 evento registrado.

Problema 19: Consolidado Mensual de Ingresos por Comisiones Bancarias

- **Objetivo de negocio:** Reportar el recaudo mensual generado por notas débito y comisiones cobradas a las cuentas del banco (Cuenta PUC 413505).
- **Tablas necesarias:** `banco_core.asiento_contable`, `banco_core.plan_cuentas`.
- **Conceptos SQL evaluados:** Agrupamiento por funciones de fecha (`DATE_TRUNC('month', fecha_contable)`), filtrado por cuenta PUC de ingresos.
- **Criterio de validación:** Mostrar el año-mes y la suma total de créditos asentados en la cuenta PUC '413505'.

Problema 20: Reconciliación de Saldos (Saldo Persistido vs. Saldo Ledger)

- **Objetivo de negocio:** Ejecutar una auditoría de integridad de datos que compare el `saldo_disponible` registrado en la tabla `cuenta` frente a la suma de sus asientos en la contabilidad.
- **Tablas necesarias:** `banco_core.cuenta`, `banco_core.transaccion`, `banco_core.asiento_contable`.
- **Conceptos SQL evaluados:** CTEs complejas, `LEFT JOIN`, aritmética de columnas, detección de descuadres (`saldo_disponible - saldo_ledger`).
- **Criterio de validación:** Devolver el ID de cuenta, número de cuenta, saldo persistido, saldo calculado según asientos contables y el valor de la diferencia.

● Nivel Avanzado (Problemas 21 a 30)

Problema 21: Ranking Regional de Clientes VIP por Departamento

- **Objetivo de negocio:** Clasificar a los clientes con mayor patrimonio dentro de su departamento de vinculación para asignación de ejecutivos de banca privada.
- **Tablas necesarias:** banco_core.cliente, banco_core.cuenta_titular, banco_core.cuenta, banco_core.oficina, banco_core.municipio, banco_core.departamento.
- **Conceptos SQL evaluados:** Funciones de ventana (DENSE_RANK() OVER (PARTITION BY ... ORDER BY ...)), CTEs.
- **Criterio de validación:** Generar una posición de ranking (1, 2, 3...) para cada cliente dentro de su departamento basado en su saldo total depositado, sin omitir puestos en caso de empates.

Problema 22: Reconstrucción del Saldo Evolutivo Diario (Running Total)

- **Objetivo de negocio:** Reconstruir la curva temporal punto a punto del saldo de una cuenta para la generación del extracto bancario detallado.
- **Tablas necesarias:** banco_core.transaccion, banco_core.asiento_contable.
- **Conceptos SQL evaluados:** Funciones de ventana con marco de acumulación (SUM(...) OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)).
- **Criterio de validación:** Para una cuenta seleccionada, listar en orden cronológico cada transacción y la columna resultante con el saldo acumulado proyectado tras cada movimiento.

Problema 23: Análisis Percentilar del Monto Transaccional (P50, P90, P99)

- **Objetivo de negocio:** Determinar los umbrales estadísticos del comportamiento transaccional para parametrización de motor de reglas antifraude.
- **Tablas necesarias:** banco_core.transaccion.
- **Conceptos SQL evaluados:** Percentiles continuos (PERCENTILE_CONT(...) WITHIN GROUP (ORDER BY ...)).
- **Criterio de validación:** Retornar en una sola fila la mediana (Percentil 50), el Percentil 90 y el Percentil 99 del monto en dinero de las transacciones en estado 'POSTED'.

Problema 24: Análisis de Cohortes de Clientes por Mes de Vinculación

- **Objetivo de negocio:** Medir la tracción comercial calculando el saldo promedio actual que mantienen los clientes según el mes y año en que se vincularon.
- **Tablas necesarias:** `banco_core.cliente`, `banco_core.cuenta_titular`, `banco_core.cuenta`.
- **Conceptos SQL evaluados:** Agrupación por cohortes temporales (`DATE_TRUNC`), agregaciones cruzadas con subconsultas de cohorte.
- **Criterio de validación:** Para cada grupo de vinculación (Año-Mes), mostrar la cantidad de clientes captados y el saldo promedio que poseen en la actualidad.

Problema 25: Detección de Transacciones Múltiples Rápidas (Patrón Rápido de Fraude)

- **Objetivo de negocio:** Identificar posibles ataques o uso indebido de tarjetas/canales detectando cuentas que realicen transferencias continuas en lapsos de tiempo muy cortos.
- **Tablas necesarias:** `banco_core.transaccion`.
- **Conceptos SQL evaluados:** Función de ventana de desplazamiento (`LAG()`), cálculo de diferencia temporal (`EXTRACT(EPOCH FROM ...)`).
- **Criterio de validación:** Retornar las transacciones donde la brecha de tiempo entre la transacción actual y la transacción inmediatamente anterior de la misma cuenta de origen sea inferior a 300 segundos (5 minutos).

Problema 26: Detección de Cuentas Inactivas Reactivadas con Transacciones Inusuales

- **Objetivo de negocio:** Detectar cuentas que permanecieron sin operaciones por más de 6 meses y repentinamente registraron una transacción de alto valor.
- **Tablas necesarias:** `banco_core.cuenta`, `banco_core.transaccion`.
- **Conceptos SQL evaluados:** `LAG()`, combinación de condiciones de tiempo (`> 180` días) y cuantía (`monto > P90`).
- **Criterio de validación:** Devolver el ID de cuenta, fecha de la transacción actual, fecha de la transacción previa, días transcurridos de inactividad y el monto de la transacción reactivadora.

Problema 27: Tasa de Crecimiento Mensual (Month-over-Month - MoM) del Volumen Transaccionado

- **Objetivo de negocio:** Entregar a la junta directiva el indicador de variación porcentual mensual del dinero movilizado por el banco.
- **Tablas necesarias:** `banco_core.transaccion`.

- **Conceptos SQL evaluados:** Aggregation con DATE_TRUNC, función de ventana LAG(), cálculo porcentual de variación $((Actual - Anterior) / Anterior) * 100$.
- **Criterio de validación:** Mostrar el Año-Mes, el volumen total transaccionado en el mes, el volumen del mes anterior y el porcentaje de variación mensual relativo.

Problema 28: Auditoría de Inconsistencias de Moneda en Transacciones y Asientos

- **Objetivo de negocio:** Verificar que ninguna transacción o asiento contable haya sido procesado utilizando una divisa diferente al Peso Colombiano ('COP').
- **Tablas necesarias:** banco_core.cuenta, banco_core.transaccion, banco_audit.auditoria_operacion.
- **Conceptos SQL evaluados:** Consultas de auditoría con LEFT JOIN y extracción de campos desde estructuras semiestructuradas JSONB.
- **Criterio de validación:** La consulta debe retornar 0 filas en un entorno conforme. Si existen inconsistencias, debe detallar las filas afectadas y los datos guardados en el payload JSONB.

Problema 29: Reconstrucción de la Línea de Tiempo Unificada de un Cliente

- **Objetivo de negocio:** Consolidar en una sola vista cronológica para la mesa de ayuda tanto los eventos administrativos de cuenta como las transacciones financieras de un cliente.
- **Tablas necesarias:** banco_core.evento_cuenta, banco_core.transaccion, banco_core.cuenta_titular.
- **Conceptos SQL evaluados:** Operador de conjuntos UNION ALL, estandarización y homologación de tipos de datos en columnas resultado, ORDER BY temporal único.
- **Criterio de validación:** Generar una lista unificada ordenada por fecha/hora ascendente con las columnas fecha, tipo_registro ('EVENTO' o 'TRANSACCION'), descripción y monto.

Problema 30: Matriz Cruzada de Flujo Monetario Origen-Destino entre Departamentos

- **Objetivo de negocio:** Construir la matriz de origen-destino de fondos entre las principales regiones comerciales de Colombia.
- **Tablas necesarias:** banco_core.transaccion, banco_core.cuenta, banco_core.oficina, banco_core.municipio, banco_core.departamento.
- **Conceptos SQL evaluados:** Pivot manual con agregación condicional (SUM(monto) FILTER (WHERE ...)) o SUM(CASE WHEN ...)), agrupamiento bidimensional.

- **Criterio de validación:** Retornar una tabla donde las filas representen el Departamento Origen y las columnas muestren los montos totales transferidos hacia 'Bogotá D.C.', 'Antioquia', 'Valle del Cauca' y 'Atlántico'.

Metodología de Evaluación para el Tutor

Cuando me envíes tu código SQL para resolver cualquiera de estos problemas, lo evaluaré bajo los siguientes **5 criterios técnicos**:

1. **Corrección Sintáctica y Semántica:** ¿El script ejecuta en PostgreSQL 16+ y responde a la pregunta de negocio sin alterar la verdad del modelo?
2. **Legibilidad y Formato:** Uso de mayúsculas en palabras clave (SELECT, FROM, WHERE), alineación de sentencias, indentación y convención `snake_case`.
3. **Eficiencia y Optimización:** Uso adecuado de índices (evitar funciones sobre columnas filtradas que fuercen *Sequential Scans*) y evitar subconsultas innecesarias cuando un JOIN o CTE resulte más óptimo.
4. **Robustez ante NULLs y Duplicados:** Uso de COALESCE(), manejo correcto de LEFT JOIN y prevención de duplicación de filas en tablas asociativas N:M (cuenta_titular).
5. **Semántica de Negocio Bancario:** Respeto por las reglas del modelo (filtrar por `id_estado_transaccion = 2` para transacciones en estado POSTED, uso de NUMERIC para dinero).